

Task 02 — Monitoring & Health Checks

The task requires you to implement:

- Actuator
- Health endpoints
- Metrics
- Application monitoring

Follow these steps in your **Spring Boot microservice**.

Step 1 — Add Actuator dependency

Open your pom.xml and add:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Save it, then in Eclipse:

Right-click Project → Maven → Update Project → Finish

Step 2 — Configure Actuator

Open:

src/main/resources/application.properties

Add:

spring.application.name=order-service

management.endpoints.web.exposure.include=health,info,metrics

management.endpoint.health.show-details=always

If your service has a different name, replace:

order-service

with your service name.

Step 3 — Test the Health Endpoint

Start your Spring Boot application.

Open your browser and enter:

<http://localhost:8080/actuator/health>

You should get:

```
{  
  "status": "UP"  
}
```

This confirms that your application is running correctly.

Step 4 — Check Application Information

Open:

<http://localhost:8080/actuator/info>

You can also add application information to application.properties:

```
info.app.name=Order Service  
info.app.description=Order microservice  
info.app.version=1.0.0
```

Then /actuator/info will expose this information.

Step 5 — Check Metrics

Open:

<http://localhost:8080/actuator/metrics>

You will see available metrics.

For example:

```
jvm.memory.used  
jvm.memory.max  
process.cpu.usage  
http.server.requests
```

To see a specific metric:

<http://localhost:8080/actuator/metrics/http.server.requests>

This helps you monitor the number and performance of HTTP requests.

Step 6 — Add a Custom Health Check

Create:

config/CustomHealthIndicator.java

Add:

```
package com.example.orderservice.config;  
  
import org.springframework.boot.actuate.health.Health;  
import org.springframework.boot.actuate.health.HealthIndicator;  
import org.springframework.stereotype.Component;  
  
@Component  
public class CustomHealthIndicator implements HealthIndicator {  
  
    @Override
```

```

public Health health() {

    return Health.up()
        .withDetail("service", "Order Service")
        .withDetail("database", "Available")
        .build();
}
}

```

Again, change the package name if your project uses a different package.

Now check:

<http://localhost:8080/actuator/health>

You should see information similar to:

```

{
  "status": "UP",
  "components": {
    "custom": {
      "status": "UP",
      "details": {
        "service": "Order Service",
        "database": "Available"
      }
    }
  }
}

```

Task 02 completed

You have implemented all four requirements:

Requirement	Implementation
Actuator	spring-boot-starter-actuator
Health endpoints	/actuator/health
Metrics	/actuator/metrics

Application monitoring Actuator metrics + health monitoring

Important URLs

<http://localhost:8080/actuator/health>

<http://localhost:8080/actuator/info>

<http://localhost:8080/actuator/metrics>